Pathfinder, for Engineers

How the Models and the Serverless Runtime Fit Together

LENSING · spotify-pathfinder (developer guide)

June 2026

Abstract

This is the engineer's tour of Pathfinder — the app that draws a 3D "musical journey" between two Spotify tracks. The interesting part is operational: it serves seven models from a *single Cloudflare Worker* with no Python origin and no vector DB at request time, by doing all the model training offline and shipping the *outputs* as a memory-mapped binary snapshot. The Rust/WASM core then does only arithmetic (search, projection) under a ∼10 ms CPU budget. We walk the architecture, the request lifecycle, the snapshot format, each model at a practical level, and how to rebuild it. Math is kept to what you need; file paths are given so you can read the source.

1 Architecture at a glance

One Worker (`worker/index.ts`) serves the static UI *and* answers `/api/*`. Everything expensive was either precomputed offline or compiled to WASM (`worker-core/`). The split that makes it fit:

- **Offline build** (Python + Qdrant + torch): train the models, score the corpus, and write three artifacts to `public/data/` — `corpus.bin`, `transitions.json`, `projector.bin`.
- **Request time** (Worker + WASM): on first hit per isolate, load the snapshot into the Rust core; thereafter every route is graph search + linear algebra over *frozen* model outputs. The only model that runs live is the cold-start text embedding (Workers AI), and only for tracks not already in the corpus.

The constraint that shapes everything. Workers Free gives ∼10 ms CPU per request. You cannot run gradient boosting or a torch encoder over 23k tracks in that budget — but you *can* memory-map a precomputed table and do a bounded A*. So the rule is: models produce **data** offline; the Worker runs **algorithms** on that data.

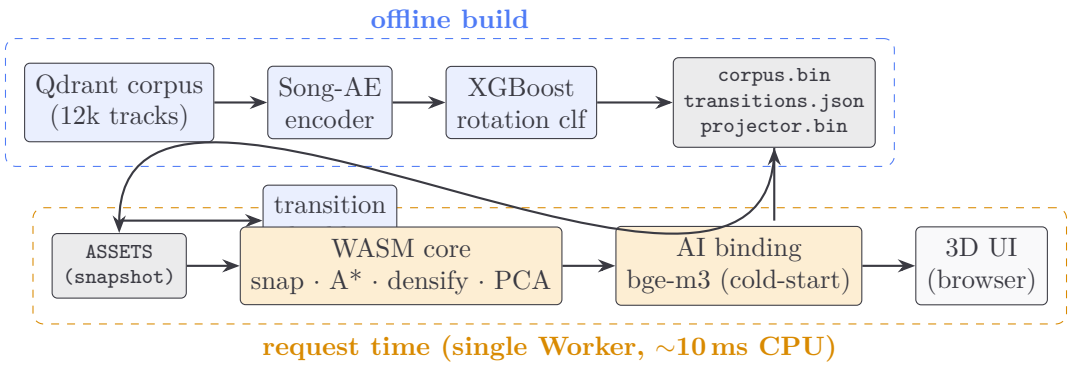

Figure 1: Train offline (blue), infer at the edge (orange). The Worker reads frozen outputs.

2 The request lifecycle

A `POST /api/route` with `{start_uri, end_uri, length, context}` flows like this (`worker/index.ts` → `worker-core/src/lib.rs`):

1. **ensureReady** (once per isolate): `initSync` the WASM module, fetch the three snapshot files via the `ASSETS` binding, `load_corpus` + `set_projector`.
2. **resolveEndpoint** for each track: `find_row(uri)`. If in corpus → use its baked vector. If not → *cold-start*: fetch Spotify/ReccoBeats metadata, embed via the AI binding, `project_and_snap` to the nearest corpus anchor.
3. **wasmRoute(startRow, endRow, ...)**: A* over the k-NN graph, `densify` to the requested length, gather a neighbor "cloud", run PCA to 3-d, return JSON (nodes with positions + per-hop "why").
4. The Worker adds snap edges for any cold-start endpoints and returns the route.

Other endpoints: `/api/search` (Spotify search, cached; falls back to corpus search), `/api/preview` (30 s clips via Deezer→iTunes), `/api/config` (public Spotify client id for the browser PKCE playlist flow).

3 The snapshot (what the Worker actually reads)

`corpus.bin` (`PFC1`) is the whole map in one memory-mapped blob: n=23,529 tracks, 64-d L2-normalized vectors, a per-track `fit` score, the k=20 neighbor graph (indices + cosine distances), and a small JSON meta block (uri/name/artist/album/genre). `transitions.json` holds the bigram/back-off/context tables. `projector.bin` (`PFP1`) holds the cold-start MLP weights + its feature config. Layouts:

PFC1 corpus.bin		PFP1 projector.bin	
field	type	field	type
magic / ver / n / dim / k	u32×5	magic … in / h / out / text	u32×6
ids	u64[n]	W_1, b_1	f32
vecs (L2-norm)	f32[n·64]	W_2, b_2	f32
fit	f32[n]	cfg_len	u32
nbr_idx / nbr_dist	u32/f32[n·k]	cfg (stats, vocab)	JSON
meta_len + JSON	u32+bytes		

Table 1: Little-endian; parsed in `snapshot.rs` / `project.rs`. Distances are baked, so a graph move at request time is a table lookup, not a dot product.

4 The models, practically

4.1 The map — Song autoencoder

An MLP autoencoder (539 → 256 → 64, trained offline) embeds each track into a 64-d space where cosine distance means "musically similar". You never run it in the Worker; you consume its output (the `vecs` in `corpus.bin`). This space is the coordinate system for the k-NN graph, the A* heuristic, and the 3D view. Its 539-d input stitches together a text embedding, numeric stats, acoustic features (+ missing flags), and genre/album one-hots (Figure 2, left).

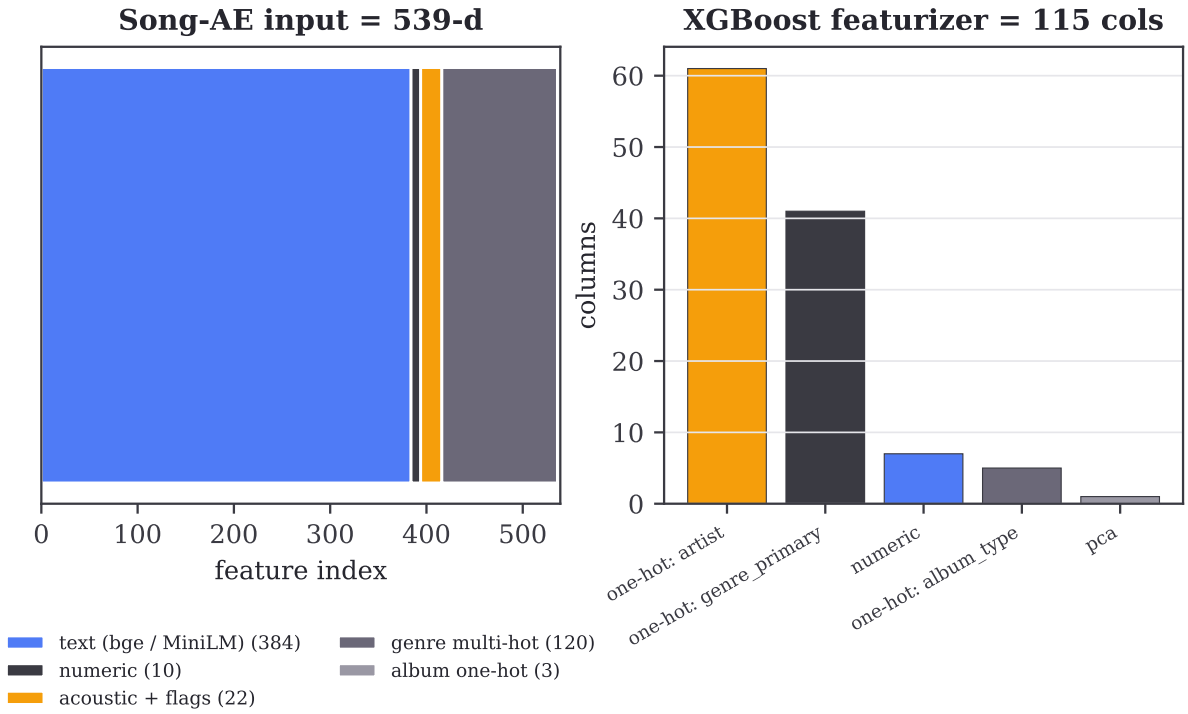

Figure 2: Left: the 539-d Song-AE input by modality. Right: the *separate* 115-column featurizer the `fit` classifier uses — mostly identity one-hots, not dense vectors.

4.2 The `fit` score — XGBoost rotation classifier

`fit` is the per-track number A* uses to prefer "sticky" tracks. It is the probability of **rotation** — `play_count >= 2`, "did the listener come back?" — from an XGBoost binary classifier

(`binary:logistic`), scored over the corpus offline, min–max normalized, and stored as the `fit` column. Test **AUC 0.7429**.

The lesson worth internalizing. The classifier's features are *identity* one-hots (artist, genre) in trees, not dense embeddings. The project tried the obvious dense features — a content-text embedding, audio features, even the Song-AE latent — and they all *hurt or diluted* the score (Figure 3). So the Song-AE latent is great as map *geometry* but a poor *feature* for the fit model; they are deliberately separate featurizers. Corollary: scan dataset/feature choices before you tune trees.

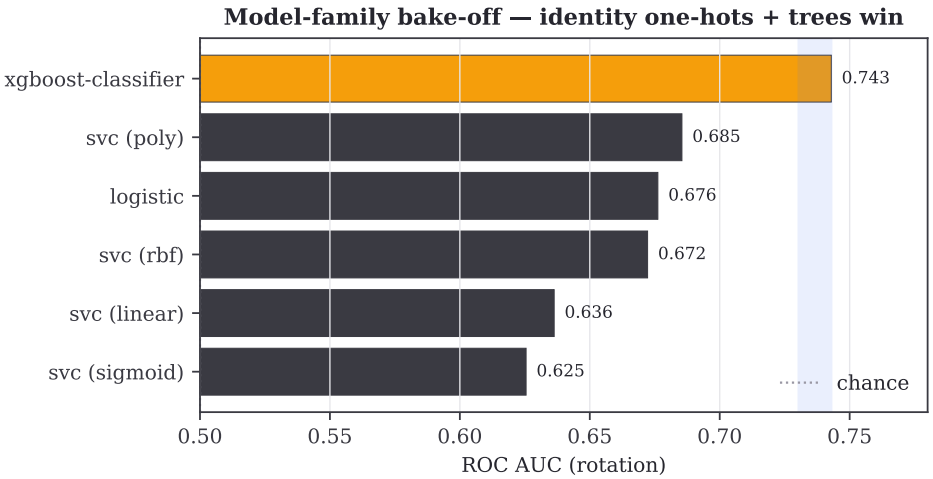

Figure 3: Real logged runs: the tree on identity one-hots beats every dense distance/margin model (SVC kernels, logistic) by several noise bands. Dense embeddings did not help the `fit` target.

4.3 Cold-start — the distilled projector

For off-map tracks you must produce a 64-d vector *in the Worker*. You can't run the real torch encoder there, so the build trains a small "projector" MLP ($1179 \to 256 \to 64$) to *reproduce* the Song-AE latents from features you can fetch live (a `bge-m3` text embedding via the AI binding + numeric/acoustic/genre/album blocks). It is a lossy translator, so the result is immediately snapped to the nearest real corpus anchor.

One detail worth copying: acoustic features from ReccoBeats are often missing. Because inputs are z-scored, an imputed mean lands at *exactly* 0 — indistinguishable from a genuinely average track. So each acoustic feature ships *two* inputs: the value and a 0/1 "was-this-real" flag. Mean-imputation made honest.

4.4 The transition model — bigram with back-off

A* also prefers transitions real sessions make. From a sessionized play log, the build forms a satisfaction-weighted track bigram, backing off to artist/genre when a track is rarely observed: it trusts the bigram in proportion to its support n_u (the count of recorded successors), $\tau = n_u/(n_u + 8)$ — think "real observations / (real + 8 prior) observations". Time-of-day and shuffle add multiplicative *lifts* (how over/under-represented a genre is in a context). Figure 4 shows the real patterns — and that the daily rhythm is a genre phenomenon, not a release-era one.

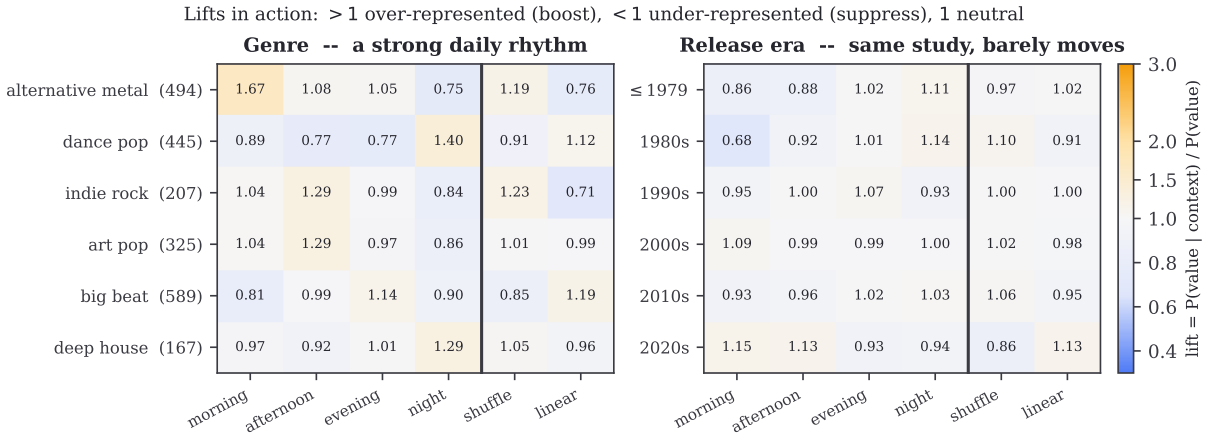

Figure 4: Real genre lifts (left) vs. the same study on release era (right). > 1 boost, < 1 suppress. Genre swings hard with time-of-day and shuffle; era barely moves.

4.5 Pathfinding — A* + densify

A* on the k-NN graph, heuristic = cosine distance to the goal. Each hop's cost is a weighted sum of five interpretable terms (no magic):

term	weight	meaning
distance	1.0	cosine step on the map
fit	0.5	prefer high-`fit` (sticky) tracks
diversity	0.5	small penalty (γ=0.15) for a genre change
transition	0.6	prefer learned next-track affinity
context	0.4	prefer time-of-day / shuffle fit

Hard constraints: no repeats, ≤ 2 same-artist in a row, no artist over $\lceil L/4 \rceil$ of an L-track route. `densify` fills short routes by inserting the best track into the widest gap. The search is capped at `MAX_EXPANSIONS = 45,000` and returns a clean 404 instead of blowing the CPU slice (a 503). The weights are constants; Figure 5 shows how the found path responds to them.

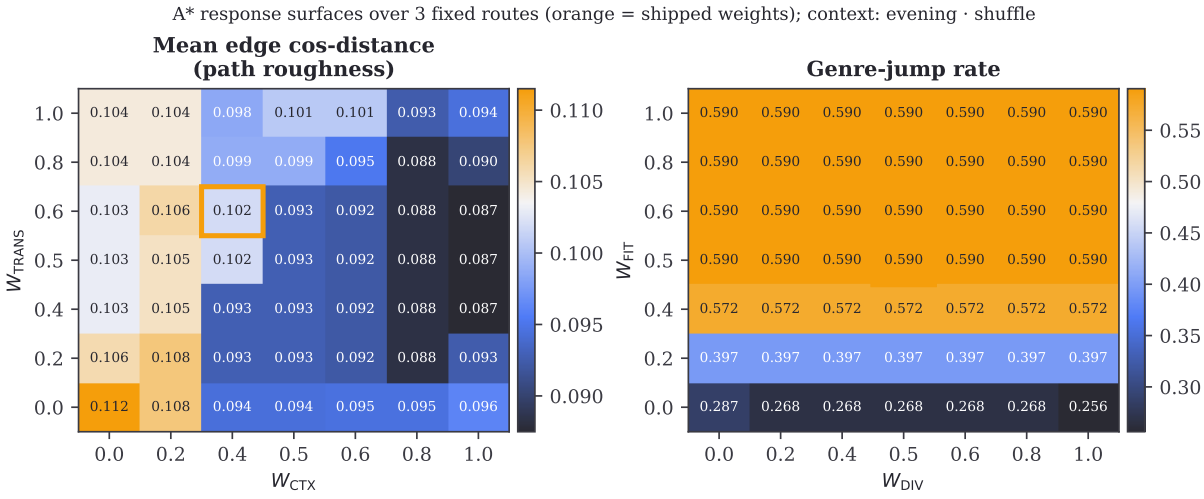

Figure 5: A* weight response surfaces over the real corpus (orange = shipped). More context weight $\Rightarrow$ smoother paths; the genre-diversity penalty is gentle.

4.6 k-NN graph and PCA

The graph is symmetric k=20 cosine neighbors, baked into `corpus.bin` (so graph moves are lookups). The 3D view projects the path+cloud from 64-d via power-iteration PCA per request — cheap because the covariance is only 64×64 over $\sim$100 points, and it avoids an SVD dependency in WASM.

5 Worked example: one route, end to end

To make the chain concrete, here is a single real request traced through every stage, with the actual numbers (regenerate with `docs/sweeps/run_example.py`). The ask: route from *Kraftwerk – "Tour de France"* to *Muse – "Intro"*, context *evening·shuffle*. Both are in the corpus, so no cold-start; the search runs on the baked graph.

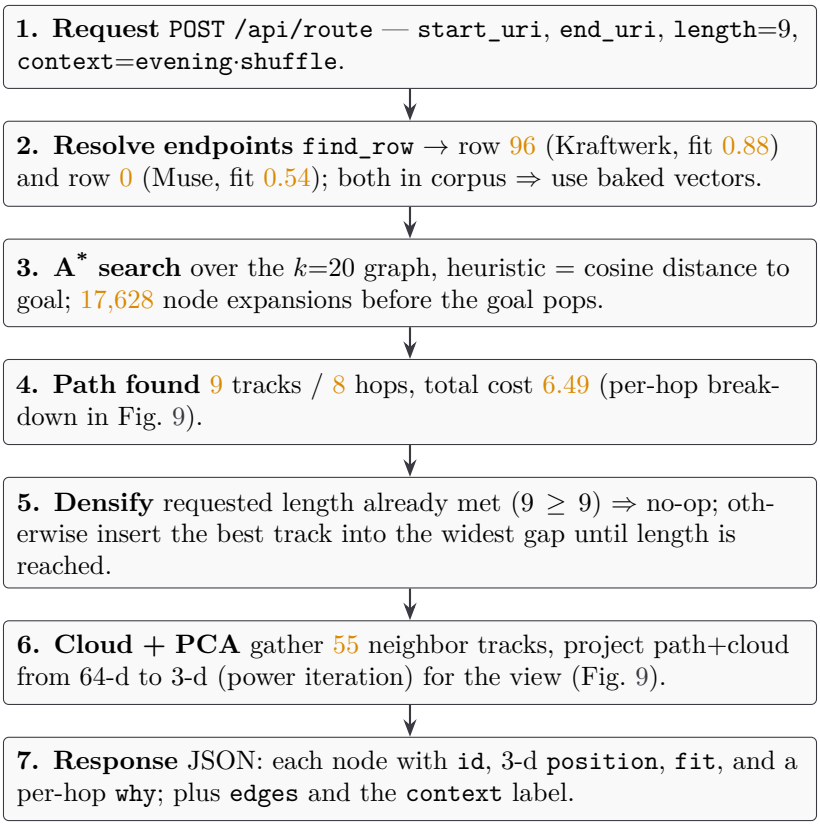

Figure 6: The full request chain for the example route, with real numbers at each stage.

The journey it returns. The nine tracks weave across genres while the straight-line distance to the Muse endpoint (h) falls steadily — that shrinking h is the heuristic pulling the search home (Figure 7).

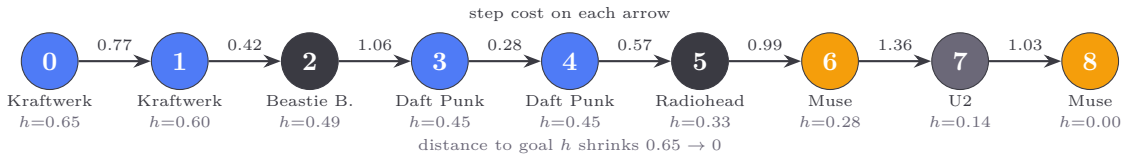

Figure 7: The route as a journey. Node colour = genre family: blue = electronic (Düsseldorf / electro), slate = alternative rock, orange = modern rock, grey = irish rock. Numbers on arrows are the per-hop A* cost; h under each node is the cosine distance still to go.

The full step-by-step (genre arc: electronic → alt-rock → electro → alt-rock → rock):

#	track / artist	genre	`fit`	step cost	dominant term
0	Tour de France — Kraftwerk	dusseldorf electronic	0.88	—	(start)
1	(Kraftwerk)	dusseldorf electronic	0.84	0.77	transition
2	(Beastie Boys)	alternative rock	0.58	0.42	fit
3	(Daft Punk)	electro	0.68	1.06	transition
4	(Daft Punk)	electro	0.87	0.28	distance
5	(Radiohead)	alternative rock	0.87	0.57	context
6	(Muse)	modern rock	0.50	0.99	transition
7	(U2)	irish rock	0.19	1.36	fit + transition
8	Intro — Muse	modern rock	0.54	1.03	transition

Table 2: The 9-track route. "Dominant term" is the largest contributor to that hop's cost. The priciest hop (7, → U2) is driven by U2's low `fit` 0.19: the fit term alone is $0.5(1 - 0.19) = 0.40$.

The decision at each step

At every track A* sees up to 20 graph neighbors (minus any blocked by the no-repeat and artist-cap constraints) and must choose one to extend toward. The key thing to understand is that it does *not* pick the locally-cheapest next step: it commits to the neighbor that lies on the cheapest *complete* route to the goal. Figure 8 shows this directly — the orange "chosen" step often sits *above* cheaper options it passed over (blue rings).

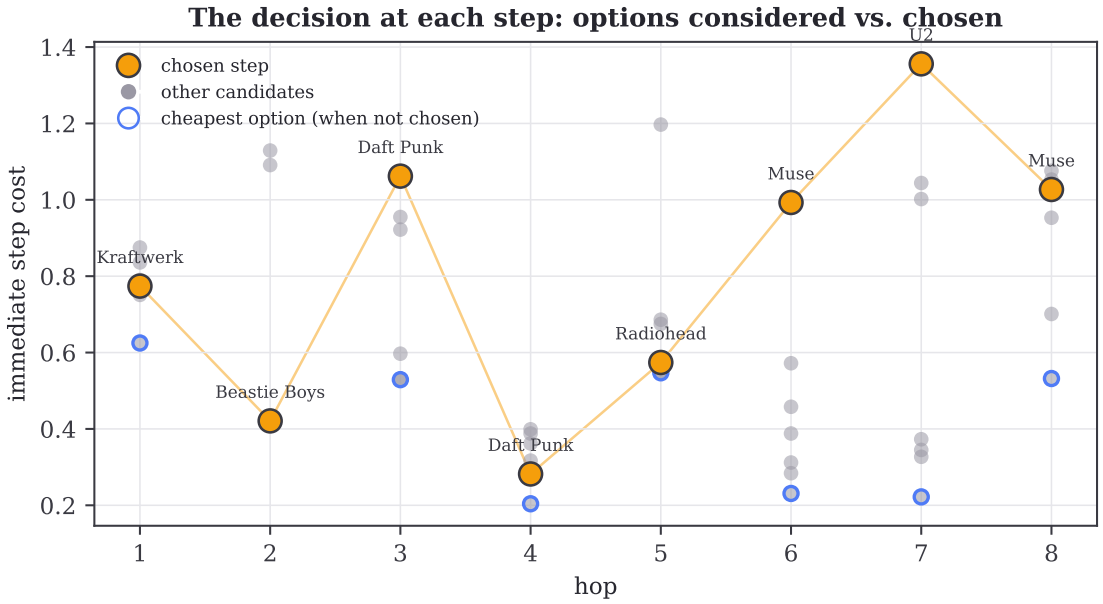

Figure 8: Every hop's menu. Grey dots are the candidate next-tracks A* weighed (by immediate step cost); orange is the one it committed to; a blue ring marks the cheapest option when that was *not* chosen. At hops 6 and 7 it deliberately pays ∼1.0–1.4 to leave a cluster, skipping ∼0.2 stay-put steps — because those cheap steps don't actually reach Muse.

Walking the decisions (Figure 7 for the path, 9 for the cost split):

- **Hop 2 — the bridge out (Kraftwerk → Beastie Boys).** The cheapest available step *and* a genre jump that drops h from 0.60 to 0.49. Easy call: leave electronic for the rock side of the map exactly when it also makes progress.
- **Hop 3 — a costly detour (→ Daft Punk, electro).** Locally a Blur step looked cheaper (0.53 vs 1.06), but A* took the pricier electro hop because it sets up a cheaper remainder — greedy would have missed it.
- **Hop 4 — the easy glide (Daft Punk → Daft Punk).** Short on the map and high-fit (0.87): the cheapest hop in the route at 0.28, an in-cluster step.
- **Hops 6–7 — leaving the cluster (→ Muse, → U2).** Six cheaper Radiohead/Muse steps (∼0.2) were on offer, but the consecutive-artist cap (max 2) and the goal pull make the jumps worth it; U2 costs the most (1.36, its `fit` is only 0.19) yet it is the bridge that brings h down to 0.14, one short hop from the Muse endpoint.

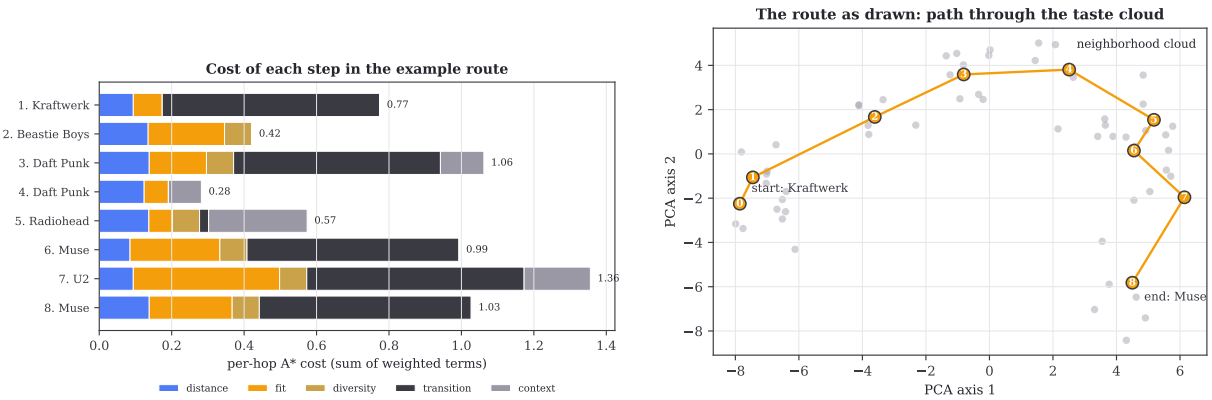

Figure 9: Left (9): each chosen hop's cost split into the five weighted terms — *why* that step cost what it did. **Right** (9): the same route as the UI draws it, threading through the grey neighborhood cloud in the PCA plane.

6 Validation

The request-time core is a Rust rewrite of Python references, so a **parity test** (`worker-core/tests/parity.rs`) checks A* against `search.py`: same endpoints/length, constraints satisfied, Rust cost $\leq 1.6\times$ Python $+0.1$, $\geq 85\%$ node identity. (Heuristic search legitimately produces alternate near-optimal paths, so exact match is the wrong bar.) A round-trip test guards the projector binary. Model deltas are judged against a measured noise band $\sigma_{\text{AUC}} = 0.0063$ — e.g. a freshly-run depth/lr scan finds a best AUC of 0.7534, only +0.011 over the shipped config, i.e. inside 2σ (Figure 10).

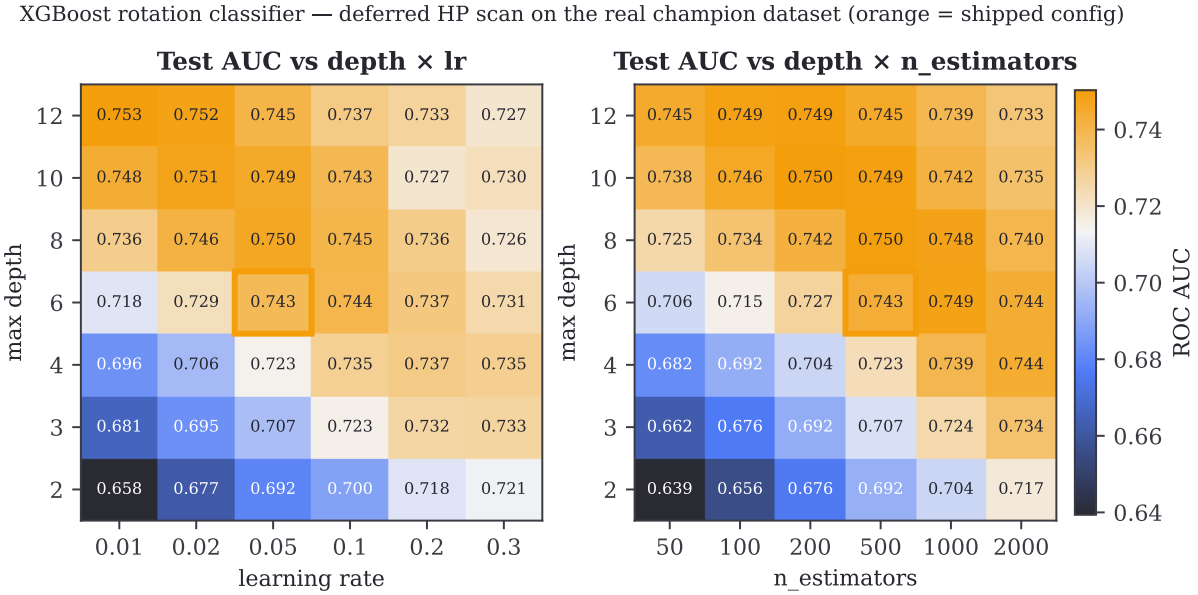

Figure 10: The deferred hyperparameter scan, run on the real dataset for this guide. Tuning moves AUC less than seed noise — don't over-invest in it.

7 Build & reproduce

```
npm run build:snapshot  # Qdrant -> public/data/* (add --projector for cold-start)
npm run build         # wasm-pack -> tsc -> vite (copies data into dist)
npm run cf:dev        # wrangler dev at 127.0.0.1:8787
```

Figures/sweeps in this guide regenerate via `docs/sweeps/*.py` + `docs/make_figures.py`; the analysis dataset and logged runs live in the sibling `spotify-predict-engagement`. For the full math (loss functions, the projector forward pass, PCA derivation, exact constants), see the companion *Models of Pathfinder* paper.